

Communication Optimization and Code Generation for Distributed Memory Machines

Saman P. Amarasinghe and Monica S. Lam
Computer Systems Laboratory
Stanford University, CA 94305

Abstract

This paper presents several algorithms to solve code generation and optimization problems specific to machines with distributed address spaces. Given a description of how the computation is to be partitioned across the processors in a machine, our algorithms produce an SPMD (single program multiple data) program to be run on each processor. Our compiler generates the necessary receive and send instructions, optimizes the communication by eliminating redundant communication and aggregating small messages into large messages, allocates space locally on each processor, and translates global data addresses to local addresses.

Our techniques are based on an exact data-flow analysis on individual array element accesses. Unlike data dependence analysis, this analysis determines if two dynamic instances refer to the same value, and not just to the same location. Using this information, our compiler can handle more flexible data decompositions and find more opportunities for communication optimization than systems based on data dependence analysis.

Our technique is based on a uniform framework, where data decompositions, computation decompositions and the data flow information are all represented as systems of linear inequalities. We show that the problems of communication code generation, local memory management, message aggregation and redundant data communication elimination can all be solved by projecting polyhedra represented by sets of inequalities onto lower dimensional spaces.

1 Introduction

Compiling to machines with a distributed address space can be separated into two major phases. The first phase determines how to decompose the computation and data across the processors. The goal is not just to parallelize the applications, but also to ensure that the programs have data locality so as to minimize the communication overhead. This step is common to all large-scale parallel machines with non uniform memory access times. If the machine has a distributed address space, then the compiler is faced with the additional problem of managing the memory and communication explicitly. The second phase of the compilation is to generate the code so that each processor will execute its allotted computation and communicate correctly and efficiently. These

algorithms can also be useful to enhance the memory system performance of machines with a shared address space.

Many systems being developed rely on the user to supply the data decompositions. Languages such as High Performance FORTRAN[24], FORTRAN-D[16] and Vienna FORTRAN[8] allow the programmer to annotate the sequential program with data decompositions. The compiler is responsible for generating the computation decomposition and an SPMD (Single Program Multiple Data) program with explicit communication.

We are developing a compiler system that automatically parallelizes a sequential program for shared and distributed memory machines. The input language to our compiler is sequential FORTRAN-77, although many of the techniques developed are also applicable to optimizations within and especially across FORTRAN-90 statements[14].

Our data and computation decomposition phase tries to maximize parallelism and minimize communication[3]. If all the available parallelism cannot be exploited without communication, this phase first tries to trade off excess degrees of parallelism to eliminate communication. The algorithm understands “doacross” parallelism, where processors are organized as a pipeline and they synchronize and communicate during the course of the computation. The algorithm will choose to use the “doacross” form of parallelism if such a choice minimizes the overall communication cost. Finally, if necessary, the algorithm will reorganize the data dynamically; it tries to insert data reorganizations into the least frequently executed parts of the program.

The decomposition phase generates a full specification of the computation decompositions. It also specifies the data decompositions before and after major data reorganizations; it does not generate data decompositions for those sections of code using “doacross” form of parallelism, where data are often allocated to different processors at different times.

Data reorganizations such as matrix transposes are implemented using collective communication routines[18]. The algorithms presented in this paper focus on generating the code and communication between reorganizations. Within each region of code requiring no major data reorganization, there may still be fine-grained communication. Sections of the arrays may be replicated and allocated to different processors at different times. The given data decompositions are used as specifications for the initial and final layouts to each code region. The algorithm uses the computation decomposition to guide the local memory management process. Within each region, our algorithm may change the data layout according to the demands of the given computation decompositions.

The data and computation decomposition schemes that a user can specify in languages like High Performance FORTRAN on dense matrices is a subset of those generated by our compiler. Thus, the techniques described in this paper are also applicable to generating code from user-supplied data decompositions.

This research was supported in part by DARPA contract DABT63-91-K-0003, an NSF Young Investigator Award and a fellowship from Intel Corporation.

Given a computation decomposition, and an initial and a final data decomposition, the techniques described in this paper automatically produce an SPMD (single program multiple data) program to be run on each processor. Our compiler generates the necessary receive and send instructions, optimizes the communication by eliminating redundant communication and aggregating small messages into large messages, allocates space locally on each processor and translates global data addresses to local addresses.

This paper presents three main results. First, we propose a *value-centric* approach to deriving the necessary communication for machines with a distributed address space. Previous approaches are *location-centric*: communication is derived from data decompositions; optimizations are performed using *data dependence*[5], an analysis that determines if accesses may refer to the same location. We derive our code generation from computation decompositions using a data-flow analysis technique that is based on values instead of locations. This approach enables a more general set of data and computation decompositions and allows for more communication optimizations.

Second, we have developed a mathematical model for code generation which is applicable to both the location- and value-centric approaches. We represent data decompositions, computations decompositions, and communication as systems of linear inequalities. We show that the various code generation and communication optimization problems can be solved by projecting the polyhedra represented by systems of inequalities onto lower dimensional spaces.

Third, we have developed several communication optimizations within the value-oriented framework. These optimizations include eliminating redundant messages, aggregating messages, and hiding the communication latency by overlapping the communication with computation. These optimizations are essential to achieving an acceptable performance on distributed memory machines[22].

The organization of our paper is as follows. In Section 2, we describe the conventional way of generating communication code from user-specified data decompositions and some of the limitations of the approach. In Section 3, we describe our value-centric approach to communication generation. We formally describe the domain of our technique and present a mathematical representation of the problem in Section 4. We describe our code generation technique and our communication optimizations in Sections 5 and 6. We finally close with a detailed example to illustrate the entire code generation process.

2 A Location-Centric Approach

Many of the existing compilers developed for distributed memory machines have a similar basic approach in how they generate code from user-specified data decompositions[4][9][16][21][22][26]. The FORTRAN-D compiler appears to be the most mature; it employs techniques such as data dependence analysis and regular array sections for communication optimizations, and substantial experimentation results have been reported. In the following, we first review the techniques used in the FORTRAN-D compiler. We then discuss some of the limitations of this approach to motivate the need for more advanced analysis and optimization.

2.1 The Basic Technique

For simplicity, in the following discussion we assume that there is only one loop nest which contains one read access and one write access to the same array. The argument obviously holds for the general case with multiple loops and arrays. There are three domains that are manipulated in the compilation process: the itera-

tion space $\mathcal{I}$, the array elements space $\mathcal{A}$, and the processor space $\mathcal{P}$. Each array access function in the source program specifies the data used by each iteration in the loop. That is, each read or write access function, denoted by $f_r, f_w: \mathcal{I} \rightarrow \mathcal{A}$ respectively, maps an iteration to the array indices of the data read or written. The user-specified data decomposition $D: \mathcal{A} \rightarrow \mathcal{P}$ maps each array location to a processor. From the read and write access functions and the data decomposition, the compiler automatically derives the computation decomposition $C: \mathcal{I} \rightarrow \mathcal{P}$ which maps each iteration in the loop to a processor.

To derive the computation decomposition, the compiler applies the *owner-computes rule*: each assignment statement is performed by the processor that owns the data. Therefore, given a write access function f_w and a data decomposition D , the computation decomposition is $C = Df_w$. Under the owner-computes rule, no communication is needed to implement the write accesses. Communication is needed for a read access in iteration i if the data read is resident on a different processor, i.e. $Ci \neq Df_r i$. The relationships between iteration space, array space, and the processor space are shown in Figure 1(a); processor p_r receives data from processor p_s if $p_r \neq p_s$.

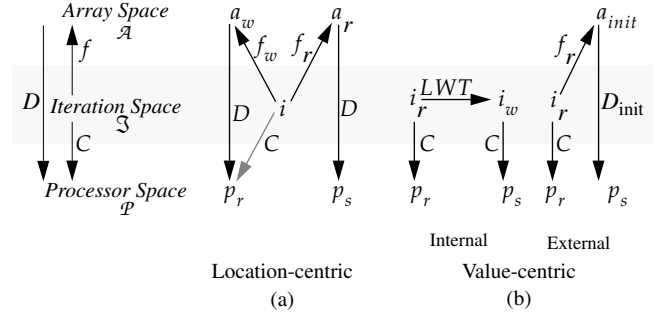


Figure 1: Different approaches to code generation for distributed memory machines.

To minimize the communication cost, the compiler tries to maximize the intervals between communication. All the data needed within the interval are sent in one message. This optimization is based on *data dependence* analysis. Two accesses are *data dependent* if one of the accesses is a write operation, and if they may refer to the same location. The dependence is carried at *level k* if the dynamic instances accessing the same location belong to the same iteration of the first $k-1$ outermost loops but not the k th loop. The *maximum level* of a dependence between two references is simply the maximum loop nest level that carries a dependence between the references. If the maximum depth of all dependences involving a read access is k , the compiler needs to communicate only once in each iteration of the k th loop. Thus, the maximum depth information is useful for reducing the communication frequency. All the data accessed within the interval requiring communication are summarized by a regular section description[15]. In this way, the same data used multiple times within the interval are only transferred once.

In summary, this approach deduces the computation decompositions from the user-specified data decompositions, using the owner-computes rule; it uses data dependence analysis to reduce the number of messages; finally, it uses the concept of regular sections to reduce redundant data transfers.

2.2 Limitations of Existing Approaches

While the approach described above can handle many regular dense matrix computations, there are still many aspects in which this approach can be improved.

2.2.1 Data Decompositions and the Owner-computes Rule

Implicit in the use of the owner-computes rule is the assumption that iterations writing to the same location must be executed by the same processor. This is unnecessarily restrictive. Consider the following example:

```
for i = 0 to n do
  for j = 1 to n do
    X[i, 0] = X[i, 0] + X[i, j]
```

Suppose the array X has been distributed as blocks of columns to facilitate the parallelization of other phases. The best way to parallelize this code is to pipeline the execution of the second loop. That is, the variable X[i,0] passes from processor to processor, and each processor contributes the sum of its section of the *i*th row to the value of X[i,0]. It is easy to express the computation decomposition of this parallelization scheme. It is not possible to achieve this scheme by specifying a data decomposition, unless we relax the owner-computes rule.

The “owner-computes” rule also assumes that the written data cannot be replicated; i.e. data decompositions map each written location to a unique address. Consider the following example:

```
for t = 0 to m do
  for i = 1 to n-1 do
    X[i] = 1/3 * (X[i] + X[i - 1] + X[i + 1])
```

A typical parallelization is to partition X into contiguous blocks, and assign to each processor the computation of one of these blocks. Since the range of data read extends beyond the block of data written, the borders of the blocks are often replicated on adjacent processors. Such data decompositions, however, would render the “owner-computes” rule inoperative, since the written data are replicated on multiple processors.

The use of the owner-computes rule plays a major role in simplifying the generation of communication code. The sender of any data is always the processor that owns the data. If we relax the owner-computes rule, deciding which processor has the desired value at any one time is much harder. If we were to replicate the written data, or move the written data dynamically, or choose not to compute at the owner’s site, we would not be able to just derive the sender of the data from the data decomposition.

2.2.2 Data Dependence Analysis

Data dependence analysis is an “alias” analysis. A read operation is dependent on a write operation as long as they may refer to the same location, even if none of the written values are used by any of the read instances[5]. The lack of value information may reduce the opportunities for parallelism and communication optimization.

Consider the following example:

```
for i = 0 to 100 do
  for j = 0 to 100 do
    work[j] = ... (S1)
  for j = 0 to 100 do
    ... = work[j] (S2)
```

The data dependence analyzer does not know if a write operation covers, or kills, an earlier write operation. Data dependence analysis will yield a level 1 dependence between statements S1 and S2 because the locations written in each outer iteration of the loop (work[0:100]) overlap with the locations read (work[0:100]). This information would serialize the outermost loop. On analyzing the flow of the data, it is obvious that data written within each iteration in the outer loop affect only the computation within the same iteration. This outer loop can be parallelized by giving each processor a private copy of the work array. This optimization, known as array privatization, has been shown to be essential to parallelize many real programs successfully[6].

Data dependence analysis does not distinguish between the different instances of the accesses. The analyzer only knows that a dependence exists; it does not know which pairs of instances are dependent. When using data dependence information for communication optimizations, the maximum depth of the dependence determines the communication interval. All the non-local data accessed within the interval must be communicated, even though some of the data may not participate in the data dependence at the same maximum depth. Thus, the same value may be transferred multiple times unnecessarily. Consider the following example:

```
for i = 0 to n do
  X[i] = ... (S1)
  for j = i to n do
    Y[j] = Y[j] + X[j - 1] (S2)
```

Suppose matrices X and Y are distributed across the processors by blocks. Using the owner-computes rule, statement S1 in iteration *i* of the outer loop is executed by the owner of X[i]; iteration *j* of the inner loop is executed by the owner of Y[j]. The data dependence between S1 and S2 indicates that communication is needed in each iteration of the outermost loop. Processors executing the second loop may need to make one non-local data reference. However, except for the processor executing the first iteration of the inner loop, none of the non-local data accessed has changed its value from the last time the loop was executed. Thus, at most one word needs to be transferred in each iteration of the outermost loop. Data dependence analysis cannot identify the instances that carry the dependence. Thus, if only data dependence information is available, we must conclude that communication is needed for every processor executing the inner loop.

Finally, compilers that use data dependence analysis to aggregate messages are limited by the resolution of the data dependence analysis. Since only the loop level information is available, the communication must necessarily take place at the iteration boundaries of the loop carrying the dependence. Potentially, if the compiler knows the precise iteration that produces the data to be communicated, we can send the data to the receiver as soon as all the data in a message have been generated. Such optimization is significant as it reduces the chances a processor is stalled waiting for data to arrive.

2.2.3 Regular sections

The contents of a message are defined by the regular section descriptor that encompasses all the non-local array elements touched between communication points. This technique may create additional traffic when the set of data accessed cannot be precisely described by a regular section. Consider the following simple example:

```
for i = 1 to 100 do
  for j = i to 100 do
    ... = A[1000 i + j]
```

Representing the data accessed as a regular section descriptor would increase the amount of communication by a factor of 20.

3 Our Approach

We have been exploring the use of a more sophisticated analysis technique that determines if two accesses refer to the same value, and not just to the same location. In the following, we describe the information obtained from the analysis and show how this can be used to eliminate the limitations of data dependence analysis discussed above.

3.1 Accurate Data-Flow Analysis

Instead of data dependence analysis, our optimizations are based on finding exact data-flow information on individual instances of array accesses. Let us use the simple example in Figure 2 to illustrate our technique.

```
for t = 0 to T do
  for i = 3 to N do
    X[i] = X[i - 3]
```

Figure 2: A simple 2-deep loop nest.

Data dependence analysis on this program will produce the dependence vectors $\{[+, 3], [0, 3]\}$, meaning that the read access in iteration $[t_r, i_r]$ may be data dependent on all iterations $[t_w, i_w]$ such that $i_w = i_r - 3$, and $t_w \leq t_r$. Perfect data flow analysis, however, is able to determine precisely that the first three iterations of the innermost loop read data defined outside the loop, and the rest of the iterations use the value defined three iterations earlier, i.e. $[t_w, i_w] = [t_r, i_r - 3]$.

The problem of finding precise array data-flow information was first formulated by Feautrier[10][11][12]. Feautrier proposed a parametric integer programming algorithm that can find such perfect information in the domain of loop nests where the loop bounds and array indices are affine functions of loop indices. We have developed an algorithm that can find the same precise information for many common cases more efficiently[19][20].

The perfect data-flow information is captured by a representation we called a *Last Write Tree* (LWT). The LWT for the above example is shown in Figure 3. The leaf M1 represents the first three iterations that read data $X[0:2]$, whose values are not generated by this program. The leaf M2 represents the rest of the iterations where the data read are written three iterations earlier.

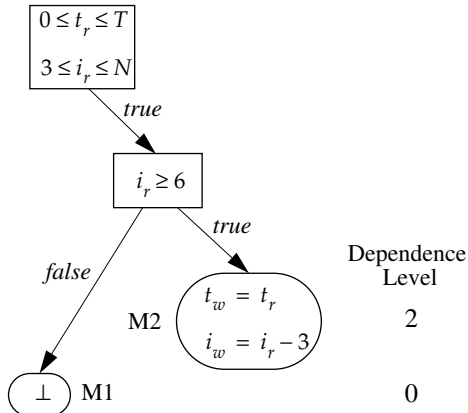


Figure 3: The Last Write Tree for the example in Figure 2.

For simplicity, our discussion below assumes that there is only one loop nest and one write access in the program. The LWT algorithm can handle multiple write operations and multiple loop nests in general.

The LWT is a function that maps an instance of a read operation to the very write instance that produces the value read, provided such a write exists. We denote a read and a write instance by the values of their loop indices, i_r and i_w , respectively. The domain of the LWT function is the set of read iterations i_r that satisfy the constraints imposed by the loop bounds. Each internal node in the tree contains a further constraint on the value of the read instance i_r . It partitions the domain of read instances into those satisfying the constraint, represented by its right descendants, and those not satisfying the constraint, represented by its left descendants. We define the *context* of a node to be the set of read iterations that satisfy the

constraints of a node's ancestors. The tree is constructed such that either all the values read within the context of a leaf are written within the loop, or none of them are. In the former case, the leaf node defines a *last-write* relation that relates each read instance i_r within the context to the write instance i_w that produces the value read. All the read-write pairs share the same dependence levels. If the instances within a context do not read any value written within the loop, we denote the lack of a writer by $\perp$.

The LWT information differs from the data dependence analysis in two major ways. First, LWTs can distinguish between the different instances of the same array access. For example, the LWT analysis can determine that the first three iterations of the inner loop in Figure 2 have different dependence relationships than all the other iterations. Second, the LWT is a function that specifies precisely the last write instance that generates the value read by a particular read instance. Data dependence analysis, on the other hand, cannot discriminate between writes to the same locations.

3.2 Using LWTs in a Distributed Memory Compiler

We have developed an array privatization algorithm based on the LWT analysis[19]. In this algorithm, parallelization is based on only the data-flow dependences generated by the LWT information. If the parallelized loop carries any anti-dependences or output dependences, then privatization is necessary. On machines with a shared address space, the compiler needs to create a private copy of the data for every processor. The private copies may need to be initialized and the final results may be written back to the original array. On machines with a distributed address space, the privatized array in a processor's local memory is no different from any other data allocated in its local memory. By generating communication only for the data-flow dependences, writes irrelevant to the computation on other processors will not affect the other processors.

Besides enhancing parallelization by enabling optimizations such as array privatization, LWTs are especially useful for efficient code generation for distributed memory machines, as explained below.

3.2.1 Deriving Communication from LWTs

There are two kinds of leaves in an LWT: $\perp$ leaves that do not read any of the values written within the code being analyzed, and leaves that do. In the former case, the compiler can simply load all the non-local data onto a processor before executing any of the code. Given a computation decomposition and an initial data decomposition produced by an earlier compiler phase, the technique to generate the necessary communication code is no different from that used in the location-centric approach.

Communication and computation are more tight-coupled for the other kind of leaves. The LWT specifies all the pairs of iterations that share a producer and consumer relationship. By applying the computation decomposition function on the related iterations, we can derive the identity of the processors that write and read the same value. If the writer and reader are different processors, then communication is necessary. Our technique is depicted in Figure 1(b).

The data decompositions generated from the earlier compiler phase serve only as interfaces with other sections of the program. In general, we generate the necessary communication from LWTs and computation decompositions, and not data decompositions. Our algorithm will change the data layout when called for by the computation decompositions. Thus, our compiler can support a wider range of data decompositions. Locations written to can be replicated or mapped to different processors across time. Furthermore, our algorithm does not rely on the restriction of the owner-computes rule.

3.2.2 Communication Optimizations

Using the LWT information, we can easily eliminate redundant data transfers. While accessing the same *location* may require multiple data transfers since the value at the location may or may not have changed, each *value* needs to be transferred once and only once. Moreover, the perfect producer and consumer information enables the compiler to issue the send immediately after the data are produced and to issue the receive just before the data are used. This maximizes the chances that the communication is overlapped with computation.

Even when a source program appears to be very simple, handling the iterations at the boundary of the loops properly can be tricky. The LWT analysis automatically partitions the read instances into sets that share similar communication characteristics. This partitioning not only makes generating correct code routine, it also enhances optimizations. For example, all iterations of each context in an LWT have the same dependence level, and read instances requiring communication within a loop are separated from those that do not. The uniformity in each of the contexts allows us to develop simple and powerful algorithms to optimize the communication. Without perfect information, the compiler may need to treat all instances uniformly and conservatively, and observe the most stringent of the requirements for all the instances.

4 Problem Formulation

In this section, we formally define the scope of our technique. We show how we can represent all the information useful for communication and computation code generation as sets of linear inequalities. This model and our techniques discussed in Section 5 are useful for both value-centric and location-centric approaches.

4.1 The Problem Domain

The scope of our technique is limited to programs consisting of a set of loop nests, where the bounds of the loop nests are affine expressions of outer loop indices and symbolic constants. The array accesses are also affine functions of loop indices and symbolic constants. Our technique can also handle conditional statements that do not contain any loops. Each assignment within the conditional statement is treated as an unconditional assignment; depending on the outcome of the condition, it assigns to the variable either the newly computed value or the variable's current value.

We can handles loops of the following form:

```

for  $i_1 = l_1(\bar{v})$  to  $h_1(\bar{v})$  do
  for  $i_2 = l_2(\bar{v}, i_1)$  to  $h_2(\bar{v}, i_1)$  do
    ...
    for  $i_n = l_n(\bar{v}, i_1, \dots, i_{n-1})$  to  $h_n(\bar{v}, i_1, \dots, i_{n-1})$  do

```

where $\bar{v}$ is a symbolic constant vector (variables unchanged within the loop) and l_k, h_k are affine functions. The iteration set of a loop is thus

$$I = \left\{ \bar{i} = (i_1, \dots, i_n) \in \mathfrak{S} \mid \forall k = 1, \dots, n \begin{array}{l} i_k \geq l_k(\bar{v}, i_1, \dots, i_{k-1}) \\ i_k \leq h_k(\bar{v}, i_1, \dots, i_{k-1}) \end{array} \right\}$$

The index set of a data array with dimensions $u_1, \dots, u_m$ is

$$A = \{ \bar{a} = (a_1, \dots, a_m) \in \mathcal{A} \mid \forall k = 1, \dots, m \ 0 \leq a_k < u_k \}.$$

The read and write access functions, $\tilde{f}_r = (f_{r_1}, \dots, f_{r_m})$ and $\tilde{f}_w = (f_{w_1}, \dots, f_{w_m})$ are affine functions of the form $f(\bar{v}, i_1, \dots, i_n) = (a_1, \dots, a_m)$,

where $(i_1, \dots, i_n) \in I$, $(a_1, \dots, a_m) \in A$ and $\bar{v}$ is a symbolic constant vector.

To support cyclic decompositions where data or computation are distributed to processors in a round-robin manner, we introduce the notion of a virtual processor array. The computation and data decompositions map the computation and data to the virtual processor space. Let $u_1, \dots, u_q$ be the dimensions of the virtual processor space, the index set of this virtual processor array is thus $P = \{ \bar{p} = (p_1, \dots, p_q) \in \mathcal{P} \mid \forall k = 1, \dots, q \ 0 \leq p_k < u_k \}.$

Our physical processor array has the same number of dimensions as the virtual processor array. Let $u'_1, \dots, u'_q$ be the physical processor array dimensions, $u'_k \leq u_k, \forall k = 1, \dots, q$. The physical processor index set is

$$P' = \{ \bar{p}' = (p'_1, \dots, p'_q) \in \mathcal{P}' \mid \forall k = 1, \dots, q \ 0 \leq p'_k < u'_k \},$$

The k th dimension of data elements or loop iterations are distributed across the physical processors in a cyclic manner whenever $u'_k < u_k$. The mapping from the virtual to the physical processor space, $\pi: \mathcal{P} \rightarrow \mathcal{P}'$ is defined as

$$\pi(\bar{p}) = \bar{p}' \text{ where } \forall k = 1, \dots, q \ p'_k = p_k \bmod u'_k.$$

Since it is only in the latter stages of the optimizations will the compiler be operating in the physical processor space, we will simply refer to virtual processors as the processors.

4.2 Data Decompositions

Definition 1: The *data decomposition relation* D is a set of array element and processor pairs $(\bar{a}, \bar{p})$, such that $(\bar{a}, \bar{p}) \in D$ iff the processor $\bar{p}$ has a copy of the array element $\bar{a}$. Data decompositions can be written as

$$D = \left\{ (\bar{a}, \bar{p}) \in A \times P \mid \begin{array}{l} U(\bar{a} - \bar{t}) \geq (\bar{b} + B\bar{u})\bar{p} - \bar{d}_l \\ U(\bar{a} - \bar{t}) < (\bar{b} + B\bar{u})(\bar{p} + 1) + \bar{d}_h \end{array} \right\}$$

where U is an extended unimodular matrix, $\bar{t}, \bar{d}_l, \bar{d}_h, \bar{b}$, are integer vectors, B is an integer matrix and $\bar{u}$ is a vector of symbolic constants such that $\bar{b} + B\bar{u} \geq 0$ and $\bar{d}_l, \bar{d}_h \geq 0$.

Our scope of data decompositions is larger than those typically used in existing distributed memory machine compilers. The matrix U determines if the array is reversed or skewed. When the array have more dimensions than that of the processor space, the 0 columns of the extended unimodular matrix U choose the dimensions to be mapped on the same processor. The entire array can be shifted with respect to the processor array using the integer vector $\bar{t}$. Since the data block size is often a function of the number of processors engaged in the computation, it is useful not to determine the block size at compile time. We can handle some symbolic block sizes of the form $\bar{b} + B\bar{u}$; the scope of our technique is discussed in Section 5.1. The overlap of array elements between processors is determined by vectors $\bar{d}_l, \bar{d}_h \geq 0$. Figure 4 illustrates how we can use this scheme to describe several common data decompositions. The 2×2 grid in each example represents the first 2×2 processors in the system; each panel is a picture of the entire data array, and the shaded portion represents the data allocated locally to that specific processor.

4.3 Computation Decompositions

Computation decompositions have a similar scope as data decompositions, except that an iteration can only be mapped onto one processor.

Definition 2: The *computation decomposition relation* C is a set of iteration and processor pairs $(i, \bar{p})$ such that processor $\bar{p}$

executes iteration $\dot{i}$ iff $(\dot{i}, \bar{p}) \in C$. Computation decompositions can be written as

$$C = \{ (\dot{i}, \bar{p}) \in I \times P \mid (\bar{b} + B\bar{u})\bar{p} \leq U(\dot{i} - \bar{i}) < (\bar{b} + B\bar{u})(\bar{p} + \bar{1}) \}$$
where U is an extended unimodular matrix, $\bar{i}$, $\bar{b}$ are integer vectors, B is an integer matrix and $\bar{u}$ is a vector of symbolic constants such that $\bar{b} + B\bar{u} > 0$.

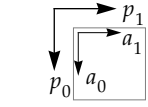
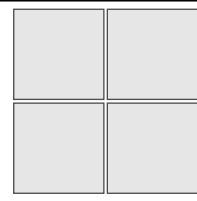
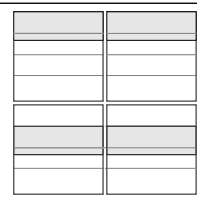
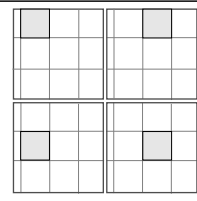
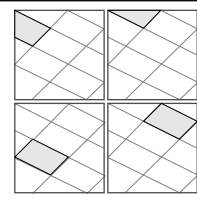
	U	$\bar{b} + B\bar{u}$	$\bar{i}$	$\bar{d}_l$	$\bar{d}_h$
	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} N \\ N \end{bmatrix}$
(a) Full replication.					
	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 15 & 0 \\ 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ N \end{bmatrix}$
(b) Blocked rows with overlap.					
	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} x & 0 \\ x & 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
(c) Square blocks with symbolic block sizes, shifted right by 1.					
	$\begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 8 & 1 \\ 16 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
(d) Skewed rectangular blocks.					

Figure 4: Examples of some data decompositions for an $N \times N$ array onto a 2-dimensional processor space.

In our compiler, computation decompositions are generated automatically by an earlier phase[3]. For systems that rely on the user to specify the data decompositions, Theorem 1 shows how to derive computation decompositions from data decompositions.

Theorem 1: Assuming that written data are not replicated, the computation decomposition as derived from data decomposition D , using the owner-computes rule, is

$$C = \{ (\dot{i}, \bar{p}) \in I \times P \mid \exists \bar{a} \in A \text{ s.t. } (\bar{a}, \bar{p}) \in D \wedge \bar{a} = \bar{f}_w(\bar{v}, \dot{i}) \}$$

4.4 Communication

We define the communication between processors formally as a communication set:

Definition 3: A **communication set** M is a set of elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in I \times P \times I \times P \times A$, where $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in M$ iff processor $\bar{p}_s$ needs to send the value in location $\bar{a}$ in iteration $\dot{i}_s$ to processor $\bar{p}_r$ for use in iteration $\dot{i}_r$.

4.4.1 Data Decompositions and the Owner-computes Rule

If we use the owner-computes rule, no communication is necessary for write operations. We use Theorem 2 to find all the necessary communication for each read access within the loop nest. We use the user-specified data decomposition to find the owner of the data read; we use the computation decomposition derived from Theorem 1 to find the processor reading the data; if these two processors are not the same, communication is needed.

Theorem 2: The communication set required by the access function f for a set of iterations $\mathfrak{v}$ under computation decomposition C and data decomposition D is the set of elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in I \times P \times I \times P \times A$,

where

$$(\dot{i}_r, \bar{p}_r) \in C, (\bar{a}, \bar{p}_s) \in D, \dot{i}_r \in \mathfrak{v}, \bar{a} = \bar{f}_r(\bar{v}, \dot{i}_r), \dot{i}_s = \dot{i}_r, \bar{p}_s \neq \bar{p}_r.$$

4.4.2 Computation Decompositions and Last Write Trees

Definition 4: An LWT partitions the iteration set of a loop nest into the **contexts** of its leaves. If the values read by the iterations in a context $\mathfrak{v} \subseteq I$ are written within the loop, then the context has a **last-write relation** μ . The last-write relation μ of the context $\mathfrak{v}$ is a set of iteration pairs $(\dot{i}_r, \dot{i}_s)$ such that $(\dot{i}_r, \dot{i}_s) \in \mu$ iff $\dot{i}_r \in \mathfrak{v}$ and $\dot{i}_s \in I$ is the iteration that generates the value read in iteration $\dot{i}_r$. A context $\mathfrak{v}$ can be written as $\{ \dot{i} \in I \mid \bar{q}(\bar{v}, \dot{i}) \geq 0 \}$ and a read-write relation μ can be written as

$$\{ (\dot{i}_r, \dot{i}_s) \in I \times I \mid \bar{q}'(\bar{v}, \dot{i}_r, \dot{i}_s) \geq 0 \},$$

where $\bar{q}$ and $\bar{q}'$ are vectors of affine expressions.

It is sometimes necessary to introduce auxiliary variables so that the last-write relations can be represented as linear inequalities. Some of the read-write relations produced by the LWT analysis algorithm are expressed as $i \equiv \beta \pmod{\alpha}$ or $i \not\equiv \beta \pmod{\alpha}$, where α, β are integers. We can introduce an auxiliary variable u , and rewrite these relations as $i - \beta = \alpha u$ and $\alpha u < i - \beta < \alpha u + \alpha$, respectively.

If a context of an LWT reads data written within the loop, then communication is needed only if the iterations sharing the read-write relation are executed by different processors (Theorem 3). If the context of an LWT uses data written outside the loop, then we use the initial data decomposition to determine the owner of the data (Theorem 4). Theorem 4 is similar to Theorem 2, except that the sends can precede the computation of the loop since the values needed are not generated within the loop.

Theorem 3: The communication set that satisfies the last-write relation μ under computation decomposition C is the set of elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in I \times P \times I \times P \times A$ where

$$(\dot{i}_r, \bar{p}_r), (\dot{i}_s, \bar{p}_s) \in C, (\dot{i}_r, \dot{i}_s) \in \mu, \bar{a} = \bar{f}_r(\bar{v}, \dot{i}_r) = \bar{f}_s(\bar{v}, \dot{i}_s), \bar{p}_s \neq \bar{p}_r.$$

Theorem 4: The communication set required by an access function f within a $\perp$ -context $\mathfrak{v}$ of an LWT, under computation decomposition C and an initial data decomposition D , is the set of elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in I \times P \times I \times P \times A$, where

$$(\dot{i}_r, \bar{p}_r) \in C, (\bar{a}, \bar{p}_s) \in D, \dot{i}_r \in \mathfrak{v}, \bar{a} = \bar{f}_r(\bar{v}, \dot{i}_r), \dot{i}_s = 0, \bar{p}_s \neq \bar{p}_r.$$

Communication decompositions, data decompositions, iteration contexts, access functions and last-write relations can all be expressed as systems of linear inequalities. The $\bar{p}_s \neq \bar{p}_r$ constraint, however, cannot be expressed as a conjunction of inequalities. We break down the inequality into a set of disjunctive conditions. For example, for a one-dimensional processor array, the constraint $p_s \neq p_r$ is represented by $p_s > p_r \vee p_s < p_r$. We represent the necessary communication as a set of communication sets, with each one satisfying all the other inequalities and one of the disjunctive conditions.

Suppose the second loop in our program in Figure 2 is distributed as blocks of 32 iterations across a linear array of processors. That is, processor p executes iteration $[t, i]$ iff

$$32p \leq \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} t \\ i \end{bmatrix} < 32(p+1)$$

(We will use this computation decomposition throughout the rest of the paper.) Figure 5 shows the communication sets for context M2 from Figure 3.

Context	$t_r \geq 0 \quad T - t_r \geq 0$ $i_r - 3 \geq 0 \quad N - i_r \geq 0$ $i_r - 6 \geq 0$ $t_s - t_r \geq 0 \quad t_r - t_s \geq 0$ $i_s - i_r + 3 \geq 0 \quad i_r - i_s - 3 \geq 0$	
Access function	$i_r - 3 - a \geq 0 \quad a - i_r + 3 \geq 0$	
Computation decomposition for read iterations	$i_r - 32p_r \geq 0 \quad 32p_r + 31 - i_r \geq 0$	
Computation decomposition for write iterations	$i_s - 32p_s \geq 0 \quad 32p_s + 31 - i_s \geq 0$	
Constraint $p_s \neq p_r$	$p_s > p_r$	$p_s < p_r$

Figure 5: Inequalities defining the communication sets for context M2 in Figure 3.

4.4.3 Finalization

Data produced within the loop nest may need to be written back to their home locations in the “final” data layout. The problem of identifying which written values are live at exit is a sub-problem in calculating last write trees[19]. The set of inequalities generated by this sub-problem, in conjunction with the final data distribution, defines the communication set for finalization.

5 Code Generation

Before we describe how to generate the computation and communication code, we first review the techniques of projection and scanning a polyhedron in Section 5.1. We then show how to generate a working program. We first generate the code for each communication set individually, merge these code fragments into an SPMD program, and manage the local memory on each processor. Optimizations are discussed in the next section.

5.1 Projection

We can represent all possible values of a set of variables $(v_1, v_2, \dots, v_n) \in Z^n$ as an n -dimensional discrete cartesian space, where the k th axis corresponds to variable v_k . Coordinate $[x_1, \dots, x_n] \in Z^n$ corresponds to the value $v_1 = x_1, \dots, v_n = x_n$.

A set of linear inequalities in $(v_1, v_2, \dots, v_n)$ defines a polyhedron S^n in this n -dimensional space. All the solutions satisfying the inequalities correspond to the integer points within the polyhedron. Suppose we project the polyhedron onto the $n-1$ dimensional subspace orthogonal to the axis representing variable v_n . The resulting polyhedron in the $(n-1)$ -dimensional subspace, S^{n-1} , can be represented by a set of linear inequalities involving $(v_1, v_2, \dots, v_{n-1})$.

If $[x_1, \dots, x_n] \in S^n$ then $[x_1, \dots, x_{n-1}] \in S^{n-1}$. However, given $[x_1, \dots, x_{n-1}] \in S^{n-1}$, there may or may not exist an x_n such that $[x_1, \dots, x_n] \in S^n$. Consider the example where S^n has a single constraint involving v_n : $v_1 = 2v_n$. We know that v_1 must be even. However, this constraint is not captured in the projected polyhedron S^{n-1} and v_1 can be an odd number in S^{n-1} .

Projection of an n -dimensional polyhedron onto an $(n-1)$ -dimensional space can be achieved using a single step of Fourier-Motzkin elimination[23]. Fourier-Motzkin elimination can produce a large number of superfluous constraints. We can determine if a constraint is superfluous as follows. We replace the constraint in question with its negation, and if the new system does not have an integer solution then the constraint is superfluous. To check if a system has an integer solution we again use Fourier-Motzkin elimination. Since the Fourier-Motzkin elimination algorithm checks if a real solution exists for a system, a branch-and-bound technique is needed to check for the existence of integer solutions[23].

We have extended the technique of projection to handle some simple non-linear inequalities so that we can handle symbolic block sizes. We allow the coefficients in the linear inequalities to be of the form $\pm(b_0 + b_1u_1 + \dots + b_mu_m)$ where $b_0, \dots, b_m \geq 0$ are integers and $u_1, \dots, u_m > 0$ are symbolic constants. The scope of our technique is limited to those cases where the result of the projection also has coefficients that are linear combinations of symbolic constants. This is an important extension because it enables the compiler to handle symbolic block sizes in data and computation decompositions; otherwise, the value of the blocks would have to be determined at compile time.

5.2 Scanning a Polyhedron

Definition 5: Given $\hat{i}, \hat{j} \in Z^n$, $\hat{i}$ is *lexicographically less than* $\hat{j}$ iff there exists $k \leq n$ such that $\forall 0 \leq l < k \quad i_l = j_l$ and $i_k < j_k$.

Given a system of linear inequalities S with unknowns $(v_1, v_2, \dots, v_n)$, we wish to create a loop nest that will enumerate all the solutions to S in lexicographic order. The desired code is in the form of an n -deep loop nest. The index of the k th outermost loop is v_k ; it is incremented by one every iteration, and has a finite lower and upper bound. The loop bounds are expressions of symbolic constants in the code and outer loop indices. This loop structure guarantees that the indices of the iterations executed are in lexicographic order. The problem that remains is: what should the bounds of the loops be such that the loop nest contains an iteration with indices $[x_1, \dots, x_n]$ iff $[x_1, \dots, x_n]$ is a solution to S ?

Ancourt and Irigoin showed that the problem can be done by a series of projections[1][2]. In the following, we briefly describe

their algorithm, then discuss a minor extension that is useful in our domain.

We find the bounds of the loop nest in reverse order. To find the bounds for v_n , we rewrite the constraints in the form of $c_l^k v_n \geq l^k(v_1, \dots, v_{n-1})$ and $c_h^k v_n \leq h^k(v_1, \dots, v_{n-1})$. Any inequalities not involving v_n need not be considered here. The integer lower and upper bounds for v_n are given simply by:

$$\text{MAX}_k \left\lceil \frac{l^k(\dots)}{c_l^k} \right\rceil \leq v_n \leq \text{MIN}_k \left\lfloor \frac{h^k(\dots)}{c_h^k} \right\rfloor$$

We next project the original polyhedron onto the $(v_1, v_2, \dots, v_{n-1})$ space, we obtain an $(n-1)$ -dimensional polyhedron represented by a set of constraints involving only variables $(v_1, v_2, \dots, v_{n-1})$. We can then repeat the process for variables $v_{n-1}, \dots, v_1$.

An example illustrating the algorithm is shown in Figure 6. In the example, we have a polyhedron defined by inequalities involving variables i and j . Steps 1 and 2 produce the bounds for the loop nest that scans the solution lexicographically in (i, j) order; whereas steps 3 and 4 produce the bounds for the loop nest that scans the solution lexicographically in (j, i) order.

As discussed in Section 5.1, there may exist an $[x_1, \dots, x_{n-1}]$ in the projected polyhedron in $(v_1, v_2, \dots, v_{n-1})$ space that does not correspond to any solution in the original system. Since all constraints involving v_n are used in the derivation of its lower and upper bounds, the lower bound of v_n will be larger than the upper bound when $v_1 = x_1, \dots, v_{n-1} = x_{n-1}$. Thus, there is a one-to-one correspondence between the iterations executed and the solutions in the system.

While the algorithm described above is correct, the generated code can be inefficient. An outer loop may contain iterations that do not have any useful computation; they simply compute the bounds of their inner loop just to find that the inner loop has no iterations. The auxiliary variables introduced by the LWT functions can often cause such inefficiency. We can usually eliminate this form of inefficiency as follows. We need not create a loop nest for v_n , if the bounds on v_n can be expressed as $\alpha v_k - \beta \leq v_n \leq \alpha v_k - \beta$, where α and β are integers. We can simply replace all references to v_n by $\alpha v_k - \beta$. If the bounds on v_n are of the form $v_k - \beta \leq \alpha v_n \leq v_k - \beta$ where $|\alpha| > 1$, we can still eliminate the loop nest for v_n by updating the bounds of v_k to:

$$\text{for } v_k = \alpha \left\lceil \frac{l - \beta}{\alpha} \right\rceil + \beta \text{ to } h \text{ step } \alpha \text{ do}$$

where l and h are the original lower and upper bounds of v_k .

5.3 Generating Computation and Communication Code

To find the computation allotted to each processor, we scan the elements in a computation decomposition relation C lexicographically in $(p_1, \dots, p_q, i_1, \dots, i_n)$, or simply $(\bar{p}, \bar{i})$, order. The $\bar{p}$ loops enumerate the processors. The inner $\bar{i}$ loops enumerate the iterations to be executed for each value of $\bar{p}$. The SPMD code to be executed by each processor is as follows. Each processor checks if its processor number is within the bounds of the $\bar{p}$ loops. If so, the code it executes is simply the $\bar{i}$ loops parameterized by its processor number. In the case where the computation decomposition is cyclic, each processor must iterate through the virtual processors it represents. Figures 7(a) and (b) show the computation code for our example in Figure 2. The rest of the figure shows the communica-

tion code for the communication sets in Figure 5. Note that no communication is necessary when $p_s > p_r$.

To generate the receive and send code for a communication set M , we scan M lexicographically in $(\bar{p}_r, \bar{i}_r, \bar{p}_s, \bar{i}_s, \bar{a})$ and $(\bar{p}_s, \bar{i}_s, \bar{p}_r, \bar{i}_r, \bar{a})$ order, respectively. In the receive loop nest, the $\bar{p}_r$ loops enumerate the processors involved in receiving data. The $\bar{i}_r$ loops specify the iterations when processor $\bar{p}_r$ needs to receive data. By definition, the $\bar{p}_s$, $\bar{i}_s$ and $\bar{a}$ loops are degenerate loops containing only one iteration. The data to be received is the value in location $\bar{a}$ on processor $\bar{p}_s$ in iteration $\bar{i}_s$. Conversely, the $\bar{p}_s$ loops in the send loop nest enumerate all the senders. The $\bar{i}_s$ loops specify the iterations when processor $\bar{p}_s$ needs to send some messages. The $\bar{p}_r$ loops identify the receivers of each message. The $\bar{i}_r$ loops specify the iterations when processor $\bar{p}_r$ need the data. The $\bar{a}$ loop is a degenerate loop containing the address of the data to be sent. If auxiliary variables have been introduced to handle modulo constraints in the LWTs, the auxiliary variables are placed last in the lexicographic order for both loops.

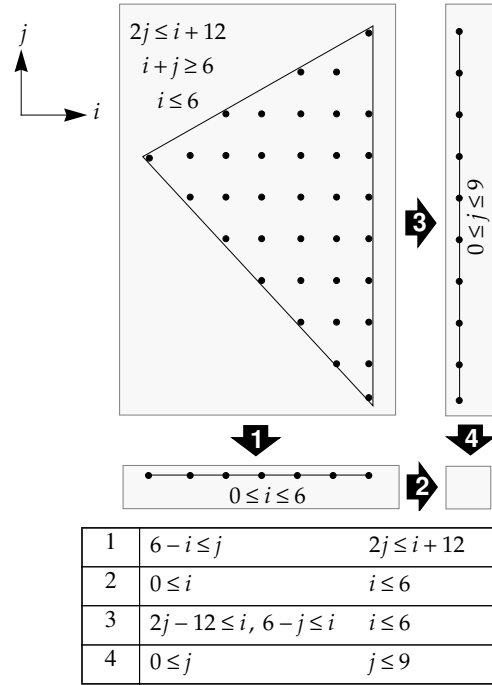


Figure 6: Example of two projection sequences on a 2-dimensional polyhedron. The table contains the bounds on the variables eliminated by each projection.

5.4 Merging Loop Nests

To generate the complete program for a processor, we need to merge together a processor's computation code, and its receive and send codes for each communication set.

A naive technique is to make each processor iterate through the entire loop nest in the source program. In each iteration, a processor checks if the iteration belongs to its computation domain, and if it is to take part in each of the communication sets. Checking such conditions in the innermost loop would be exorbitantly expensive. Some of this inefficiency can be eliminated by standard data-flow optimizations such as algebraic simplification, invariant code motion, strength reduction and common subexpression elimination.

Since all the conditions tested are affine expressions, we can potentially eliminate all run-time checks by splitting loops. Suppose we need to merge the following loops:

<pre> if p >= 0 and p <= N / 32 then for t = 0 to T do for i = MAX(32 p, 3) to MIN(32 p + 31, N) do X[i] = X[i - 3] </pre> (a) Computation code: scanning C in (p, t, i) order.
<pre> for p_v = p_p to N / 32 step P do for t = 0 to T do for i = MAX(32 p_v, 3) to MIN(32 p_v + 31, N) do X[i] = X[i - 3] </pre> (b) Computation code when virtual processors p_v are mapped to P physical processors (p_p).
<pre> if p_r >= 1 and p_r <= N / 32 then for t_r = 0 to T do for i_r = 32 p_r to MIN(32 p_r + 2, N) do p_s = p_r - 1 t_s = t_r i_s = i_r - 3 a = i_r - 3 receive X[a] from iteration (t_s, i_s) in processor (p_s) </pre> (c) Receive code: scanning M2 in (p_r, t_r, i_r, p_s, t_s, i_s, a) order.
<pre> if p_s >= 0 and p_s <= N / 32 - 1 then for t_s = 0 to T do for i_s = 32 p_s + 29 to MIN(32 p_s + 31, N - 3) do p_r = p_s + 1 t_r = t_s i_r = i_s + 3 a = i_s send X[a] to iteration (t_r, i_r) in processor (p_r) </pre> (d) Send code: scanning M2 in (p_s, t_s, i_s, p_r, t_r, i_r, a) order.

Figure 7: Computation and communication code for the example in Figure 2.

```

for i = 0 to 200 do
  receive(...)
for i = 100 to 300 do
  send(...)

```

Instead of generating:

```

for i = 0 to 300 do
  if 0 <= i and i <= 200 then
    receive(...)
  if 100 <= i and i <= 300 then
    send(...)

```

we can generate:

```

for i = 0 to 99 do
  receive(...)
for i = 100 to 200 do
  receive(...)
  send(...)
for i = 201 to 300 do
  send(...)

```

We have developed an algorithm that merges multiple nested loops together using the technique of loop splitting. Details of the algorithm are beyond the scope of this paper. If the relative magnitude between the bounds of the individual loops is not known at compile time, loop splitting can expand the program size by a significant amount. Our compiler only uses loop splitting on inner loops, and also when the relative magnitudes between the loop bounds

are known. We have also developed a dynamic splitting scheme that we use on the outer loops. The compiler does not generate all the possible combinations statically. Instead, each processor determines its bounds for all the iteration sets, sorts the bounds, and interprets the sorted list to determine the loops it has to execute. Finally, for iteration sets that are a function of only outermost loop variables, we insert dynamic checks into the bodies of the outer loops.

5.5 Local Address Space

Typically, a processor on a parallel machine touches only a part of an array. Since data sets processed by these programs are often very large, it is essential that the compiler only allocates, on each processor, storage for the data used by the processor.

The following is a simple approach to the memory allocation problem. We allocate on each processor the smallest rectangular region that covers all the data read or written by the processor, and we copy all the received data from the communication buffer to their home locations in the array before they are accessed. Given a computation decomposition C and an access function f, the set of locations touched by processor p is

$$\{\bar{a} \in A \mid \exists \bar{i} (\bar{i}, \bar{p}) \in C \wedge a = \bar{f}(\bar{v}, \bar{i})\}.$$

By scanning the inequalities lexicographically in $(\bar{p}, a_k, \bar{i})$ order, the bounds we obtain on a_k are the bounds for the kth dimension of the bounding box covering access f. If there are multiple accesses to the same array, we simply find the bounding box of the rectangular boxes for all the accesses to the same array. Note that this formulation allows local data spaces on different processors to overlap.

The above algorithm is inadequate if the rectangular bounding box of the data accessed is larger than the available local memory on the processor, while the data actually used fit in the local memory. Also, a processor's local memory may not be large enough to fit all the data that a processor will eventually use in a computation. In that case, we need to manage the memory dynamically.

The LWT information provides a more efficient way to manage the data that have been received from other processors. The compiler knows precisely which values are read by every instance of the read access. Instead of first copying all the received data to their home locations, a processor can simply read the values directly from the communication buffers. The compiler also has the information on when the buffer is no longer needed, and can manage the buffer space effectively.

6 Optimizations

Since the above algorithms generate a receive and a send message for every read access to remote data, the code is correct but inefficient. It is essential that we eliminate the redundant messages and amortize the message sending overhead by batching the communication.

6.1 Eliminating Redundant Communication

Ancourt has also studied the problem of eliminating redundant communication[2]. Given a set of iterations and accesses, Ancourt's algorithm can construct a set of loop nests that fetches all the data touched without any duplication. This algorithm is adequate for removing redundant traffic if no communication is required within the loop nest. In general, transfers of data with the same address are redundant only if the values transferred are identical.

We separate redundancy into two categories. We say that there is *self reuse* when multiple instances of a single read access use the same data and *group reuse* when instances of different read accesses use the same data. We discuss each of these in turn below.

6.1.1 Redundant Communication Due to Self Reuse

Read instances that have different data-flow relationships often are amenable to different communication optimizations. By partitioning the read instances into different contexts according to their data-flow patterns, the LWT makes it easier to detect and eliminate redundancy. Our algorithm applies Theorem 5 to the communication set of each context to detect redundancy due to self reuse.

Theorem 5: Given a communication set M , communications $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a})$, $(\dot{i}'_r, \bar{p}'_r, \dot{i}'_s, \bar{p}'_s, \bar{a}')$ $\in M$ are *redundant due to self reuse* if $\bar{p}_r = \bar{p}'_r$, $\bar{p}_s = \bar{p}'_s$, $\dot{i}_s = \dot{i}'_s$ and $\bar{a} = \bar{a}'$.

All elements in a communication set with identical $\dot{i}_s$, $\bar{p}_s$ and $\bar{a}$ refer to the same values; all elements with identical $\dot{i}_s$, $\bar{p}_s$, $\bar{p}_r$ and $\bar{a}$ are redundant messages. Thus, we wish to replace the set of redundant messages with $(\min(\dot{i}_r), \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a})$. This can be achieved by projecting the set onto the $(\bar{p}_s, \dot{i}_s, \bar{p}_r, \bar{a})$ space, and constraining the upper bound of $\dot{i}_r$ to be identical to its lower bound. There are two complications. First, if the lower bound of $\dot{i}_r$ is expressed as a conjunction of multiple inequalities involving outer loop indices, then the communication set containing the minimum $\dot{i}_r$'s is no longer convex. The algorithm needs to divide the communication set into multiple convex sets. The second complication arises from the fact that a projected image may contain points that do not correspond to a solution to the original system. In many cases, a simple test can determine that no such degeneracies are present[19].

6.1.2 Redundant Communication Due to Group Reuse

Detection of reuse between arbitrary accesses to the same matrix can be expensive. However, there is a prevalent form of reuse that can be incorporated and exploited easily within our model and that is the set of *uniformly generated references*[13]. Array index functions of uniformly generated references are affine functions of loop indices and symbolic constants, and they differ only in the constant terms. For example, $X[i]$ and $X[i+3]$ are uniformly generated references; so are $B[2i+3j+1, 3j+n+3]$ and $B[2i+3j+10, 3j+n+2]$, but not $C[i]$ and $C[j]$. Reuse between uniformly generated references has been exploited successfully in improving cache locality[27][28]. Uniformly generated references are quite common in real programs, so much so that specialized languages and compilers have been built to translate them to efficient code[7][17].

We can represent a set of uniformly generated references by their convex hull, and describe the data flow information for all the references by a single LWT. For example, suppose the example in Figure 2 has three more read accesses:

```
for t = 0 to T do
  for i = 3 to N do
    X[i] = f(X[i], X[i - 1], X[i - 2], X[i - 3])
```

Figure 8: The example in Figure 2 with multiple read accesses.

The set of accesses $X[i]$, $X[i - 1]$, $X[i - 2]$, $X[i - 3]$ can be represented by the access function $f(i)$, where $f(i) = i - u$ and $0 \leq u \leq 3$. The LWT for all the accesses is given in Figure 9.

Note that the convex hull may contain more data than those accessed within an iteration; however, since a processor is typically responsible for a contiguous block of iterations, this method is unlikely to cause any significant unnecessary traffic.

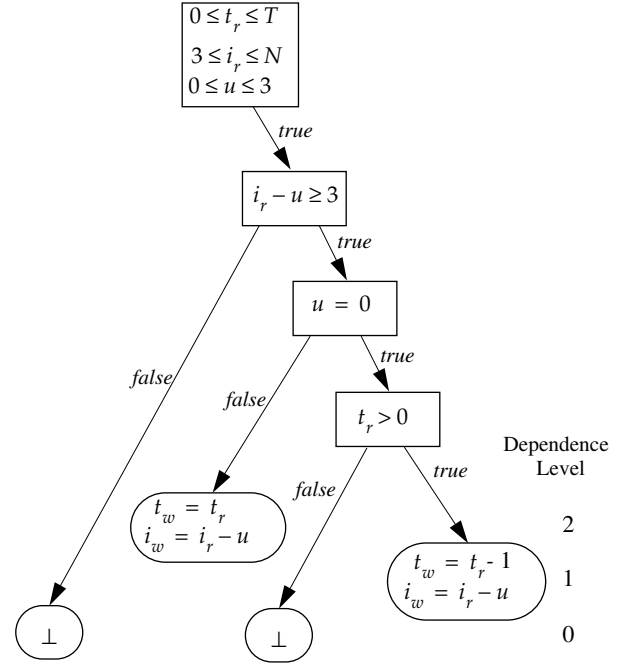


Figure 9: The Last Write Tree of the example in Figure 8.

6.1.3 Other forms of redundancies

Redundancy may also arise from cyclic decompositions, where a physical processor emulates multiple virtual processors. Given a virtual to physical processor mapping $\pi: \mathcal{P} \rightarrow \mathcal{P}$, communication $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in M$ can be eliminated if $\pi(\bar{p}_r) = \pi(\bar{p}_s)$. Also, communications $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a})$, $(\dot{i}'_r, \bar{p}'_r, \dot{i}'_s, \bar{p}'_s, \bar{a}')$ $\in M$ are redundant if $\pi(\bar{p}_r) = \pi(\bar{p}'_r)$, $\bar{p}_s = \bar{p}'_s$, $\dot{i}_s = \dot{i}'_s$ and $\bar{a} = \bar{a}'$.

Communication sets derived from data decompositions that replicate data may also contain redundancy. In our definition of communication sets, we consider communication to be necessary as long as there is a processor that owns a copy of the data needed by another processor. That means communication is generated even if the processor already owns a copy of the data. To eliminate this redundancy, we eliminate all the communication elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a}) \in M$ such that $(\bar{a}, \bar{p}_r) \in D$. Furthermore, two communication elements $(\dot{i}_r, \bar{p}_r, \dot{i}_s, \bar{p}_s, \bar{a})$, $(\dot{i}'_r, \bar{p}'_r, \dot{i}'_s, \bar{p}'_s, \bar{a}')$ $\in M$ are redundant due to replicated data if $\bar{p}_r = \bar{p}'_r$, $\bar{p}_s \neq \bar{p}'_s$, $\dot{i}_s = \dot{i}'_s$ and $\bar{a} = \bar{a}'$. The technique to eliminate this redundancy is similar to that of removing redundant communication due to self-reuse.

6.2 Communication Aggregation

Whether aggregation of small messages into large messages is necessary depends on the machine architecture. For example, machines such as the iWarp and CM-5 support fine-grain communication, while machines such as the Intel iPSC have significant overhead in processing every message. Again, we classify message aggregation into two kinds: *self aggregation*, where messages generated by different instances of the same access are aggregated, and *group aggregation*, where messages generated by different accesses are aggregated. For group aggregation, we simply aggregate all messages that have the same sender, receiver and dependence level into one message.

While group aggregation reduces the number of messages by a small constant, self aggregation can potentially eliminate many more messages. Our self aggregation algorithm also takes advantage of the partitions created by the LWT analysis. All instances

within the same communication set have the same dependence level. If the dependence level of a communication set is k , it is obviously legal to batch all the messages within an iteration of loop k and send the data at the end of the iteration. This can result in significant overhead reduction if loop k is not the innermost loop.

The algorithm to aggregate the communication of a communication set at level k is as follows. To generate the send code, we scan the communication set lexicographically in

$$(\bar{p}_s, i_{s_1}, \dots, i_{s_{k-1}}, \bar{p}_r, i_{r_1}, \dots, i_{r_n}, \bar{a})$$

order. Each instance of the loops $\bar{p}_s, i_{s_1}, \dots, i_{s_{k-1}}, \bar{p}_r$ produces one message, and each instance of the loops $i_{s_k}, \dots, i_{s_n}$ contributes an item to the message. Redundancy elimination would have caused i_r to take on only the value of the earliest iteration on the receiver side using the value. Similarly, we create the receive loop nest by scanning the polyhedron in $(\bar{p}_r, i_{r_1}, \dots, i_{r_{k-1}}, \bar{p}_s, i_{s_1}, \dots, i_{s_n}, \bar{a})$ order. Iterations in loops $i_{s_k}, i_{r_k}, \dots, i_{r_n}$ use the data from the same message. Note that for each message, the order in which the sender packs the data is the same as the unpacking order. Figure 10 shows the receive and send code for the M2 context in Figure 3 after aggregation.

```

if  $p_r \geq 1$  and  $p_r \leq N / 32$  then
  for  $t_r = 0$  to  $T$  do
     $p_s = p_r - 1$ 
    receive data into buffer from processor  $p_s$ 
     $index = 0$ 
    for  $i_s = 32 p_r - 3$  to  $\text{MIN}(32 p_r - 1, N - 3)$  do
       $t_s = t_r$ 
       $i_r = i_s + 3$ 
       $a = i_r - 3$ 
       $X[a] = \text{buffer}[index]$ 
       $index = index + 1$ 

if  $p_s \geq 0$  and  $p_s \leq (N - 32) / 32$  then
  for  $t_s = 0$  to  $T$  do
     $p_r = p_s + 1$ 
     $index = 0$ 
    for  $i_s = 32 p_s + 29$  to  $\text{MIN}(32 p_s + 31, N - 3)$  do
       $t_r = t_s$ 
       $i_r = i_s + 3$ 
       $a = i_s$ 
       $\text{buffer}[index] = X[a]$ 
       $index = index + 1$ 
    send the data in buffer to processor  $p_r$ 

```

Figure 10: Aggregated communication for context M2 in Figure 3.

6.2.1 Multi-casting

Many systems provide optimized routines for multi-casting. To take advantage of these routines, we need to determine if the same message is sent to multiple processors. We scan a communication set to be aggregated at level k lexicographically in $(\bar{p}_s, i_{s_1}, \dots, i_{s_{k-1}}, \bar{p}_r, \bar{a}, i_{s_k}, \dots, i_{s_n}, \bar{i}_r)$ order. If the bounds of $\bar{a}$ are independent of $\bar{p}_r$, the data sent to each processor are identical.

7 A Detailed Example

We have implemented the communication optimization and code generation algorithms described in this paper in the Stanford SUIF compiler system[25]. However, these algorithms have not been fully integrated with all the other phases in the compiler. We have

been able to experiment with our algorithms on the Intel iPSC/860 machine by doing the following. We enter the decompositions manually. From the decompositions, our compiler pass generates all the necessary communication and computation loop nest structures and their bounds in C. We then insert in the original loop body and compile the program using the vendor's C compiler. In this section, we show how our algorithm compiles and optimizes the LU decomposition kernel in Figure 11.

```

for  $i_1 = 0$  to  $N$  do
  for  $i_2 = i_1 + 1$  to  $N$  do
     $X[i_2][i_1] = X[i_2][i_1] / X[i_1][i_1]$ 
    for  $i_3 = i_1 + 1$  to  $N$  do
       $X[i_2][i_3] = X[i_2][i_3] - X[i_2][i_1] * X[i_1][i_3]$ 

```

Figure 11: Sequential version of LU decomposition.

To improve load balance, we use a cyclic computation and data decomposition in this program. The computation decomposition C and initial data decomposition D first map the computation and data, respectively, to a virtual processor space, which is later mapped to the physical processor space. Let $\vec{i} = (i_1, i_2, i_3)$, the decompositions used are

$$C = \{ (i, p) \in I \times \mathcal{P} \mid p \leq \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \vec{i} < p + 1 \}$$

and

$$D = \{ (\vec{a}, p) \in A \times \mathcal{P} \mid p \leq \begin{bmatrix} 1 & 0 \end{bmatrix} \vec{a} < p + 1 \}$$

That is, the k th virtual processor executes the k th iteration of the second loop and owns the k th row of array X .

First, our algorithm generates the LWTs for all the five read accesses in the program. Every tree has two leaves, one $\perp$ leaf and one leaf reading from either the $X[i_2][i_3]$ or $X[i_2][i_1]$ write operation. The LWT for the read access $X[i_1][i_3]$ is shown in Figure 12.

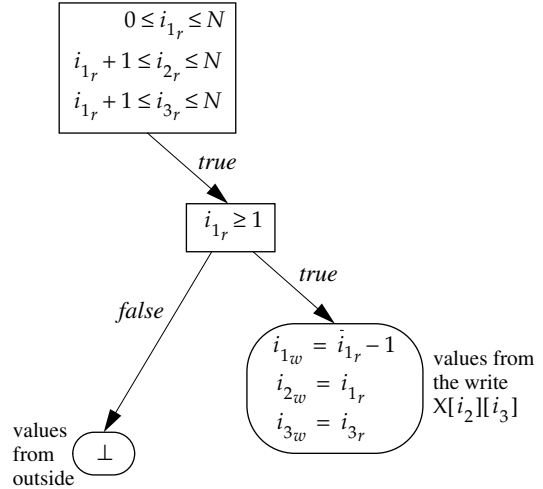


Figure 12: The LWT for the read access $X[i_1][i_3]$.

Second, our compiler calculates the communication set for every leaf of each tree. It determines that both leaves on the LWTs for $X[i_1][i_3]$ and $X[i_1][i_1]$ have non-empty communication sets. All other leaves need no communication, either because the values read are produced by the same processor or they are already in the local memory due to the initial data decomposition.

The communication sets specify the related read and write dynamic instances precisely. We know precisely that it is the first iteration of the second loop that produces all the data used by $X[i_1][i_3]$ and $X[i_1][i_1]$ in the next iteration of the outermost loop. Knowing this information enables us to issue the send immediately after the first iteration of the second loop. If we had used only data dependence analysis, we would have to delay the send until all the

iterations of the second loop have been executed. Sending the data early maximizes the potential for overlap between communication and computation.

Next, the compiler performs several communication optimizations. It first aggregates all the messages within each of the two communication sets of $X[i_1][i_3]$. Then, the communication sets for the $\perp$ leaves of both $X[i_1][i_3]$ and $X[i_1][i_3]$ are aggregated. The same is done for the other two leaves. At this point, each virtual processor receives only a single message in each iteration of the outermost loop. The compiler then discovers that the message contents are independent of the identity of the receiving processor, thus the data can be multi-cast. Since the computation decomposition is cyclic, the compiler optimizes the communication further by sending the data only to one virtual processor in each physical processor.

Lastly, the compiler decides on how to manage the local memory. While each virtual processor always writes to the same row in the matrix, every processor would have read the entire matrix by the end of the computation. The compiler creates a local array for the write accesses, and uses a buffer to hold the data a processor receives. The program uses the data in the buffer directly. The dimensions of the local array on each physical processor are $((N+P)/P) \times 1 \times (N+1)$ where P is the number of physical processors in the system. Note that the middle dimension of this local array can be trivially eliminated. The communication buffer is of size $N+1$, the size of the largest aggregated message.

Figure 13 is a pretty-printed version of the information generated by the compiler. In this code, the variable “myp” identifies the physical processor executing the code. Most of the loop overhead present in this code can be eliminated using classical optimizations.

```
float X[(N + P)/P][N+1], B[N+1]
{ First processor sends initial data }
if N >= 1 and myp = 0 then
  for t = 0 to N do
    B[t] = X[0][t]
  for pr = 1 to MIN(P - 1, N) do
    send B of size N + 1 to (pr mod P)
for i1 = 0 to N - 1 do
  { iterate over the virtual processor set }
  for p = P*((i1 + 1 - myp + P - 1)/P) + myp to N step P do
    i2 = p
    c = i2/P          { local address }
    { receive data, once for each physical processor }
    if i1 >= i2 - P + 1 then
      receive to B of size N - i1 + 1
    X[c][i1] = X[c][i1] / B[0]
    for i3 = 1 + i1 to N do
      X[c][i3] = X[c][i3] - X[c][i1]*B[i3 - i1]
    { send data }
    if i1 <= N - 2 and i2 = i1 + 1 then
      for t = i2 to N do
        B[t - i2] = X[c][t]
      for pr = i2 + 1 to MIN(i2 + P - 1, N) do
        send B of size N - i2 + 1 to (pr mod P)
```

Figure 13: Compiler generated SPMD code for LU decomposition.

Our compiler pass took 2.9 seconds to generate the computation and communication code. Our 2048 × 2048 LU-decomposition code runs at 250 MFLOPS (single-precision) on a 32-processor

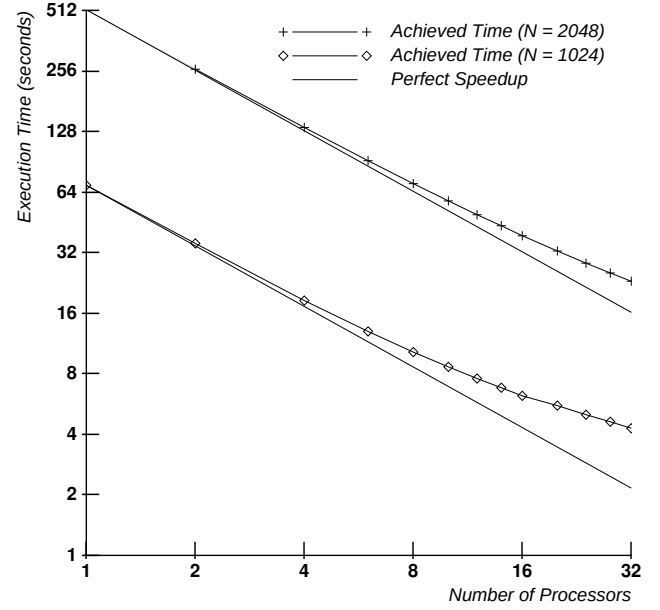


Figure 14: Performance for a single-precision LU decomposition on an Intel iPSC/860.

Intel iPSC/860 machine. Figure 14 shows the performance of our code for two matrix sizes on different numbers of processors.

8 Summary and Conclusion

This paper introduces a value-centric approach to communication generation for machines with a distributed address space. We use a precise data-flow analysis that determines the exact value read by every instance of a read access; we derive communications from computation decompositions. This approach has several advantages over the more conventional approach of deriving communications from data decompositions, and optimizing communications using data dependence analysis. Our approach supports more flexible data decomposition schemes: data may not have a static home, data may be replicated, and the owner-computes rule can be relaxed. Our approach can also support more communication optimizations. The LWT analysis automatically partitions the read instances into sets that share similar communication characteristics. This simplifies the handling of boundary conditions and also exposes more opportunities for parallelism.

We have developed a uniform framework to solve the problems of communication code generation, local memory management, message aggregation and redundant data communication elimination. We represent data decompositions, computation decompositions and the data flow information all as systems of linear inequalities. The various code generation and communication optimization problems can be solved by projecting polyhedra represented by sets of inequalities onto lower dimensional spaces. Our techniques are applicable to both the value-centric approach presented in this paper as well as the conventional location-centric approach used in other systems.

Acknowledgments

We would like to thank Jennifer Anderson, Amy Lim, Dror Maydan and Daniel Scales for their contributions to this work. We acknowledge the support of the SUIF compiler group, and Robert Dutton and Dan Yergeau for providing and supporting the iPSC/

860 machine. Finally, we thank Mary Hall, Amy Lim, Martin Rinard, Prabhat Samarutunga and Bob Wilson for providing invaluable comments on the paper.

References

- [1] C. Ancourt and F. Irigoin. Scanning Polyhedra with DO Loops. In *Third ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP*, pp. 39-50, April 1991.
- [2] M. Ancourt. *Generation automatique de codes de transfert pour multiprocesseurs a memoires locales*. PhD thesis, University of Paris VI, March 1991.
- [3] J. M. Anderson and M. S. Lam. Global Optimizations for Parallelism and Locality on Scalable Parallel Machines. In *Proceedings of the SIGPLAN '93 Conference on Program Language Design and Implementation*, June 1993.
- [4] F. André, O. Chéron and J.-L. Pazat. Compiling Sequential Programs for Distributed Memory Parallel Computers with Pandore II. In *Third Workshop on Compilers for Parallel Computers*, pp. 231-242, July 1992.
- [5] U. Banerjee. *Dependence Analysis for Supercomputing*. Kluwer Academic, 1988.
- [6] W. Blume and R. Eigenmann. Performance Analysis of Parallelizing Compilers on the Perfect Benchmarks Programs. In *IEEE Transactions on Parallel and Distributed Systems*, vol. 3, no. 6, pp. 643-656, November 1992.
- [7] M. Bromley, S. Heller, T. McNeerney and G. L. Steele Jr. Fortran at Ten Gigaflops: The Connection Machine Convolution Compiler. In *Proceedings of ACM Conference on Programming Language Design and Implementation*, pp. 145-164, June 1991.
- [8] B. Chapman, P. Mehrotra, and H. Zima. Programming in Vienna Fortran. In *Third Workshop on Compilers for Parallel Computers*, pp. 121-160, July 1992.
- [9] C. Koelbel. Compile-time generation of regular communication patterns. In *Proceedings of Supercomputing '91*, pp. 101-110, November 1991.
- [10] P. Feautrier, Array expansion. In *International Conference on Supercomputing*, pp. 429-442, 1988.
- [11] P. Feautrier, Parametric integer programming. Technical Report 209, Laboratoire Methodologie and Architecture Des Systems Informatiques, January 1988.
- [12] P. Feautrier. Dataflow analysis of array and scalar references. In *International Journal of Parallel Programming*, 20(1):23-52, February 1991.
- [13] G. Gannon, W. Jalby and K. Gallivan. Strategies for cache and local memory management by global program transformation. In *Journal of Parallel and Distributed Computing*, 5:587-616, 1988.
- [14] G. R. Gao, R. Olsen, V. Sarkar and R. Thekkath. Collective loop fusion for array contraction. In *Proceedings of the Fifth Workshop on Programming Languages and Compilers for Parallel Computing*, pp. 171-181, August 1992.
- [15] P. Havlak and K. Kennedy. An implementation of interprocedural bounded regular section analysis. In *IEEE Transactions on Parallel and Distributed Systems*, vol. 2, no. 3, pp. 350-360, July 1991.
- [16] S. Hiranandani, K. Kennedy and C. Tseng. Compiling Fortran D for MIMD Distributed-Memory Machines. In *Communications of ACM*, vol. 35, no. 8, pp. 66-80, August 1992.
- [17] L. G. C. Hamey, J. A. Webb and I. C. Wu. An architecture independent programming language for low-level vision. *Computer Vision, Graphics, and Image Processing*, vol. 48, no. 2, pp. 246-264, November 1989.
- [18] J. Li and M. Chen. Index domain alignment: Minimizing cost of cross-referencing between distributed arrays. In *Proceedings of Frontiers '90: The Third Symposium on the Frontiers of Massively Parallel Computation*. pp. 424-432. October 1990.
- [19] D. E. Maydan, S. P. Amarasinghe and M. S. Lam. Array Data-Flow Analysis and its Use in Array Privatization. In *Proceedings of ACM Conference on Principles of Programming Languages*, pp. 2-15. January 1993.
- [20] D. E. Maydan, *Accurate Analysis of Array References*. PhD thesis, Stanford University, September 1992. Published as CSL-TR-92-547.
- [21] P. Mehrotra and J. Van Rosendale. High Level Programming of Distributed Memory Architectures. In A. Nicolau, D. Gelernter, T. Gross and D. Padua editors, *Advances in Languages and Compilers for Parallel Processing*, pp. 364-384, The MIT Press, Cambridge, Massachusetts, 1991.
- [22] K. Pingali and A. Rogers. Process decomposition through locality of reference. In *Proceedings of the SIGPLAN '89 Conference on Program Language Design and Implementation*, pp. 69-80, June 1989.
- [23] A. Schrijver, *Theory of Linear and Integer Programming*, Wiley, Chichester 1986.
- [24] G. L. Steele. Proposal for alignment and distribution directives in HPF. Draft presented at HPF Forum meeting, June 1992.
- [25] S. Tjiang, M. Wolf, M. Lam, K. Pieper and J. Hennessy. Integrating scalar optimizations and parallelization. In U. Banerjee, D. Gelernter, A. Nicolau and D. Padua editors, *Languages and Compilers for Parallel Computing*, pp. 137-151, Springer-Verlag, Berlin, Germany, 1992.
- [26] C.-W. Tseng. *An Optimizing Fortran D Compiler for MIMD Distributed-Memory Machines*. PhD thesis, Rice University, January 1993. Published as Rice COMP TR93-199.
- [27] M. E. Wolf and M. S. Lam. A data locality optimizing algorithm. In *SIGPLAN Notices*, vol. 26, no. 6, pp. 30-44, June 1991.
- [28] M. E. Wolf. *Improving locality and parallelism in nested loops*. PhD thesis, Stanford University, August 1992. Published as CSL-TR-92-538.